

Data Structures Notes: Trees

1. Introduction to Trees

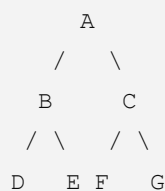
A **Tree** is a non-linear data structure used to represent hierarchical relationships between elements.

Unlike arrays, stacks, queues, and linked lists, where data is generally organized sequentially, a tree organizes data in **levels**.

Real-life examples

- File and folder systems
- Organization hierarchy
- HTML/XML document structure
- Database indexing
- Expression evaluation
- Artificial intelligence decision trees
- Network routing

Basic Tree Example

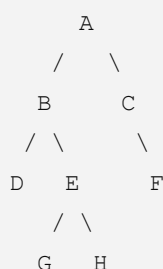


Here:

- **A** is the root.
- **B** and **C** are children of **A**.
- **D, E, F, G** are leaf nodes.

2. Basic Tree Terminologies

Consider the following tree:



2.1 Node

A **node** is an individual element of a tree.

A

Here, A is a node.

A node generally contains:

```
+-----+-----+
|  Data  | Link |
+-----+-----+
```

2.2 Edge

An **edge** is the connection between two nodes.

A
|
B

The connection between A and B is an edge.

If a tree contains n nodes, it always contains:

$n - 1$ edges

2.3 Root

The **root** is the topmost node of a tree.

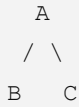
```
    A      ← Root
   /  \
  B    C
```

Here, A is the root.

A tree has **exactly one root node**.

2.4 Parent

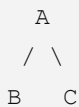
A node that has one or more children is called a **parent**.



A is the parent of **B** and **C**.

2.5 Child

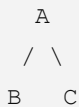
A node directly connected below another node is called its **child**.



B and **C** are children of **A**.

2.6 Siblings

Nodes having the same parent are called **siblings**.



B and **C** are siblings.

In the larger tree:



D and **E** are siblings.

2.7 Leaf Node

A node having **no children** is called a **leaf node**, terminal node, or external node.



D and **C** are leaf nodes.

2.8 Internal Node

A node having at least one child is called an **internal node** or non-leaf node.



A and **B** are internal nodes.

2.9 Degree of a Node

The **degree of a node** is the number of children it has.



Degree of **A** = **3**

Degree of **B** = **0**

2.10 Degree of a Tree

The **degree of a tree** is the **maximum degree of any node** in the tree.



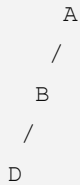
Maximum number of children of a node = 3.

Therefore:

Degree of tree = 3

2.11 Path

A **path** is a sequence of nodes connected by edges.



Path from **A** to **D**:

A → B → D

Number of edges in this path = 2.

2.12 Path Length

The number of edges in a path is called its **path length**.

A → B → D → G

Path length = 3

2.13 Level of a Node

The level represents the **position of a node relative to the root.**

One common convention is:

Level 0:	A
Level 1:	B C
Level 2:	D E F G

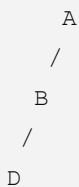
Thus:

- $A \rightarrow \text{Level } 0$
- $B, C \rightarrow \text{Level } 1$
- $D, E, F, G \rightarrow \text{Level } 2$

Note: Some textbooks define the root as Level 1. Always follow the convention specified in your course.

2.14 Depth

The **depth** of a node is the number of edges from the root to that node.



Depth:

- $A = 0$
- $B = 1$
- $D = 2$

2.15 Height of a Node

The height of a node is the number of edges on the longest downward path from that node to a leaf.

For example:



Height of:

- $D = 0$
- $E = 0$
- $B = 1$
- $C = 0$

- $A = 2$
-

2.16 Height of a Tree

The height of a tree is the height of its root.



Longest path:

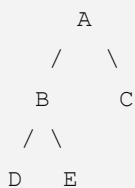
$A \rightarrow B \rightarrow C \rightarrow D$

Therefore:

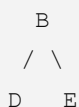
Height = 3 edges

2.17 Subtree

A **subtree** is a tree formed by a node and all of its descendants.

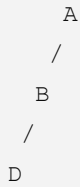


Subtree rooted at B :



2.18 Ancestor

A node is an **ancestor** of another node if it occurs on the path from the root to that node.

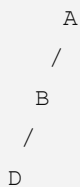


Ancestors of **D** :

B, A

2.19 Descendant

A node below another node is its descendant.



Descendants of **A** :

B, D

3. Types of Trees

There are many types of trees:

Different types of Trees:

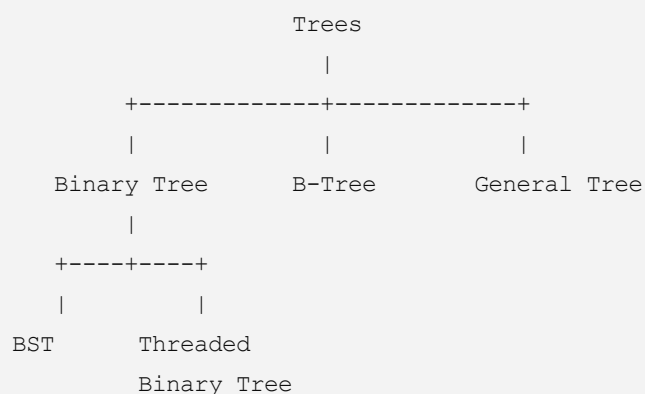
Binary Tree

Binary Search

Tree

AVL Tree

Threaded Binary Tree



In this section, we focus on:

1. Binary Tree
2. Threaded Binary Tree

3. Binary Search Tree (BST)

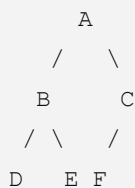
4. Binary Tree

A **Binary Tree** is a tree in which each node can have **at most two children**.

The two children are called:

- Left child
- Right child

Example



Every node has at most two children.

Binary Tree Node Structure

In C, a binary tree node can be represented as:

C

```
struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
};
```

Diagram:

```
+-----+-----+-----+  
| Data | Left | Right |  
+-----+-----+-----+
```

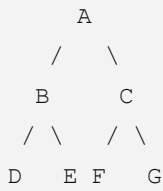
5. Important Types of Binary Trees

5.1 Full Binary Tree

A **Full Binary Tree** is a binary tree in which every node has either:

- 0 children, or
- 2 children.

Example:



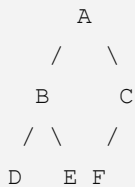
Every internal node has exactly two children.

5.2 Complete Binary Tree

A **Complete Binary Tree** is a binary tree in which:

- All levels except possibly the last are completely filled.
- Nodes at the last level are filled from **left to right**.

Example:



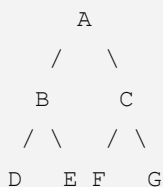
The last level is filled from left to right.

5.3 Perfect Binary Tree

A **Perfect Binary Tree** has:

- Every internal node with **exactly two children**.
- **All leaf nodes at the same level**.

Example:



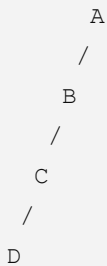
For a perfect binary tree of height h :

Number of nodes = $2^{(h+1)} - 1$

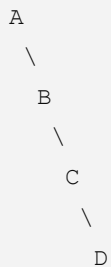
5.4 Skewed Binary Tree

A skewed tree is a binary tree in which nodes mostly have only one child.

Left-skewed



Right-skewed



A skewed binary tree behaves similar to a linked list.

6. Binary Tree Traversals

Traversal means **visiting every node of a tree exactly once**.

The major depth-first traversals are:

1. Preorder
2. Inorder
3. Postorder

Consider:





6.1 Preorder Traversal

Order:

Root → Left → Right

For the above tree:

A → B → D → E → C → F → G

Algorithm

```
PREORDER(root)
  if root != NULL
    visit(root)
    PREORDER(root->left)
    PREORDER(root->right)
```

6.2 Inorder Traversal

Order:

Left → Root → Right

For the above tree:

D → B → E → A → F → C → G

Algorithm

```
INORDER(root)
  if root != NULL
    INORDER(root->left)
    visit(root)
    INORDER(root->right)
```

6.3 Postorder Traversal

Order:

Left → Right → Root

For the above tree:

D → E → B → F → G → C → A

Algorithm

```
POSTORDER(root)
    if root != NULL
        POSTORDER(root->left)
        POSTORDER(root->right)
        visit(root)
```

Traversal Summary

For:

```
      A
     / \
    B   C
   / \
  D   E
```

Traversal	Order
Preorder	A B D E C
Inorder	D B E A C
Postorder	D E B C A

7. Threaded Binary Tree

Problem with an Ordinary Binary Tree

Consider:



A binary tree node has two pointer fields:

```

left
right

```

Many of these pointers may contain `NULL`.

For example, leaf `D` has:

```

left  = NULL
right = NULL

```

Threaded Binary Trees use some of these `NULL` pointers to store useful links called **threads**.

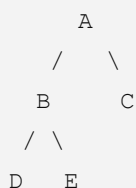
Definition

A **Threaded Binary Tree** is a binary tree in which `NULL` child pointers are replaced by special links called **threads**, usually pointing to the node's predecessor or successor in a traversal order.

The most common form is based on **inorder traversal**.

Example

Ordinary binary tree:



Inorder:

```

D → B → E → A → C

```

Threads can be used to connect nodes according to this inorder sequence.

For example:

```
D → B  
E → A  
C → NULL
```

These links can help perform inorder traversal without recursion or an auxiliary stack.

Types of Threaded Binary Trees

1. Left Threaded Binary Tree

NULL left pointers are replaced by links to the **inorder predecessor**.

2. Right Threaded Binary Tree

NULL right pointers are replaced by links to the **inorder successor**.

3. Fully / Double Threaded Binary Tree

Both NULL left and NULL right pointers can be used as threads.

```
Left thread → inorder predecessor  
Right thread → inorder successor
```

Threaded Node Structure

A threaded binary tree generally needs additional information to identify whether a pointer is a child or a thread.

```
C  
  
struct Node {  
    int data;  
    struct Node *left;  
    struct Node *right;  
    int lthread;  
    int rthread;  
};
```

For example:

```
lthread = 1 → left pointer is a thread
lthread = 0 → left pointer is a child

rthread = 1 → right pointer is a thread
rthread = 0 → right pointer is a child
```

Advantages

- Inorder traversal can be performed without recursion.
- No separate stack may be required for traversal.
- NULL pointers are utilized.
- Finding inorder successor/predecessor can be efficient.

Disadvantages

- Implementation is more complicated.
- Insertion and deletion require careful thread maintenance.
- Additional flags are generally required.

8. Binary Search Tree (BST)

A **Binary Search Tree** is a binary tree that follows an ordering property.

For every node:

```
Values in left subtree
    <
    Root
    <
Values in right subtree
```

More precisely, under the common distinct-key convention:

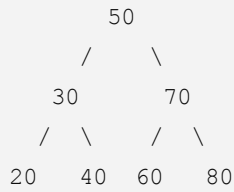
```
Left subtree < Node < Right subtree
```

Example

Insert:

```
50, 30, 70, 20, 40, 60, 80
```

BST becomes:



Check:

20 < 30 < 40 < 50 < 60 < 70 < 80

9. BST Searching

Suppose we want to search for:

60

Start at root:

60 > 50

Go right.

60 < 70

Go left.

60 = 60

Found.

Path:

50 → 70 → 60

BST Search Algorithm

```
SEARCH(root, key)
```

```
1. If root == NULL
```

```
        return NOT FOUND

2. If key == root->data
    return FOUND

3. If key < root->data
    SEARCH(root->left, key)

4. Otherwise
    SEARCH(root->right, key)
```

10. BST Insertion

To insert a value:

1. Start from the root.
2. Compare the value with the current node.
3. If smaller, move left.
4. If larger, move right.
5. Continue until an empty position is found.

Example

Insert:

50, 30, 70, 20, 40

Step-by-step:

Insert 50

50

Insert 30:

50
/
30

Insert 70:

50
/ \
30 70

Insert 20:



Insert 40:



11. BST Deletion

Deletion has three important cases.

Case 1: Delete a Leaf Node

Example:



Delete 20 .

Result:



Case 2: Delete a Node Having One Child

Before:

```

      50
     /
    30
   /
  20

```

Delete 30 .

20 takes the position of 30 .

```

      50
     /
    20

```

Case 3: Delete a Node Having Two Children

Consider:

```

      50
     / \
    30  70
     / \
    60  80

```

Delete 70 .

We can replace it with its **inorder successor**, which is the smallest node in its right subtree.

Inorder successor of 70 = 80

Result:

```

      50
     / \
    30  80
     /
    60

```

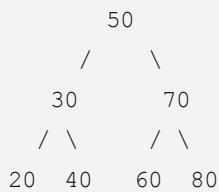
Alternatively, the inorder predecessor can be used.

12. Inorder Traversal of BST

One of the most important properties of a BST:

Inorder traversal of a BST produces the keys in sorted order under the usual distinct-key convention.

Example:



Inorder:

20 → 30 → 40 → 50 → 60 → 70 → 80

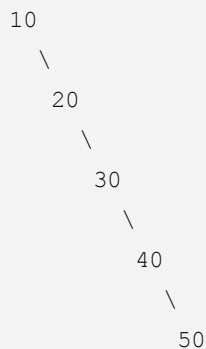
Therefore, inorder traversal is useful for obtaining BST elements in ascending order.

13. Time Complexity of BST Operations

For a reasonably balanced BST:

Operation	Average Case
Search	$O(\log n)$
Insertion	$O(\log n)$
Deletion	$O(\log n)$

However, in the worst case, a BST can become skewed:



It then behaves similarly to a linked list.

Worst-case complexity:

Operation	Worst Case
Search	$O(n)$
Insertion	$O(n)$
Deletion	$O(n)$

14. Binary Tree vs BST

Feature	Binary Tree	Binary Search Tree
Maximum children	2	2
Ordering rule	No specific ordering required	Left < Root < Right*
Searching	Generally $O(n)$	Average $O(\log n)$
Inorder traversal	Not necessarily sorted	Sorted order*
Main use	Hierarchical representation	Searching and ordered data
Example	Expression tree	Dictionary/search structure

*Assuming the conventional distinct-key BST definition.

15. Binary Tree vs Threaded Binary Tree

Feature	Binary Tree	Threaded Binary Tree
NULL pointers	Usually remain NULL	Can be used as threads
Traversal	Often recursion/stack	Can traverse using threads
Extra flags	Not normally required	Usually required
Implementation	Simpler	More complex
Memory utilization	NULL links are unused	NULL links can store useful traversal links

Feature	Binary Tree	Threaded Binary Tree
Main advantage	Simple representation	Efficient traversal without recursion/stack

16. Important Exam Points

Remember these definitions

Tree:

A non-linear hierarchical data structure consisting of nodes connected by edges.

Binary Tree:

A tree in which each node has at most two children.

Threaded Binary Tree:

A binary tree in which NULL child pointers are utilized as threads to connect nodes according to a traversal order, commonly inorder.

Binary Search Tree:

A binary tree in which the left subtree contains smaller keys and the right subtree contains larger keys, under the standard distinct-key convention.

Key formulas

For a tree with n nodes:

$$\text{Number of edges} = n - 1$$

For a binary tree of height h , when root is at height 0:

$$\text{Maximum nodes at level } h = 2^h$$

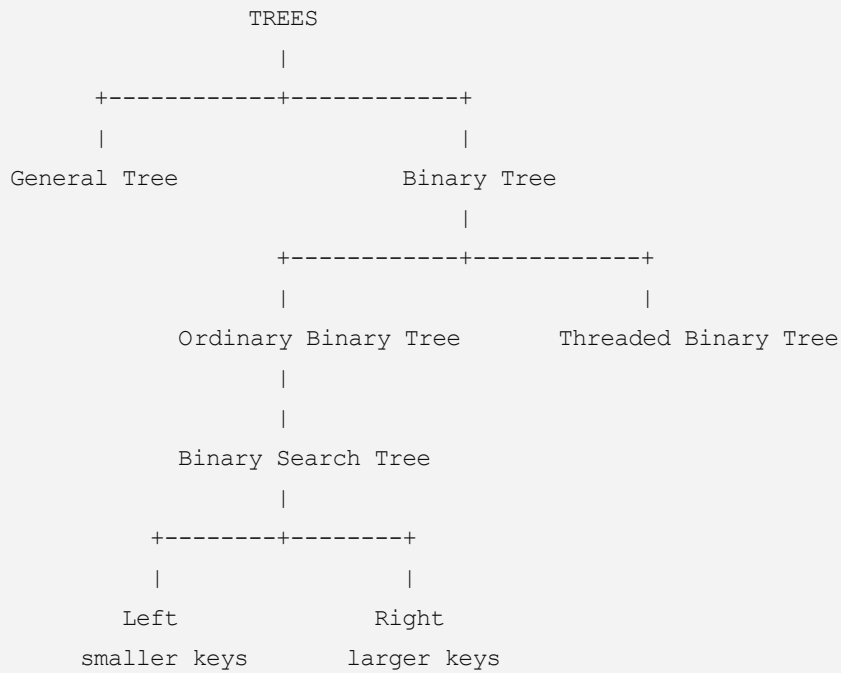
Maximum number of nodes in a binary tree of height h :

$$2^{(h+1)} - 1$$

For a perfect binary tree with n nodes:

$$\text{Number of leaf nodes} = (n + 1) / 2$$

17. Quick Revision Diagram



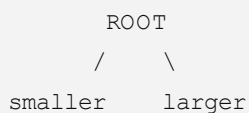
Traversals to remember

PREORDER = Root → Left → Right

INORDER = Left → Root → Right

POSTORDER = Left → Right → Root

BST rule



Most important relationship:

Binary Tree



Can have at most 2 children

BST



Binary Tree + Ordering Property

Threaded Binary Tree



Binary Tree + Threads replacing NULL links